International Journal of Information Security (2025) 24:154
https://doi.org/10.1007/s10207-025-01067-3

REGULAR CONTRIBUTION

Evaluating the Impact of Adversarial Patch Attacks on YOLO Models and the Implications for Edge AI Security

D. L. Gala[1] · J. Molleda[2] · R. Usamentiaga[2]

Accepted: 22 May 2025 / Published online: 7 June 2025
© The Author(s) 2025

Abstract
Machine learning (ML) and deep learning methods have demonstrated exceptional performance in diverse domains, including computer vision, speech and face recognition, autonomous vehicles, the Internet of Things, and cybersecurity. Despite these advances, the vulnerability of ML systems to adversarial attacks, which introduce perturbations to fool classifiers and detectors, poses significant security challenges. As the artificial intelligence (AI) industry increasingly adopts edge computing, the security of AI models becomes critical. Edge devices, constrained by limited computational resources, must run smaller models to achieve comparable latency to cloud-based models on more powerful servers. This work evaluates the impact of adversarial attacks-malicious perturbations designed to degrade the accuracy and reliability of ML models-on recent YOLO detectors from the widely used Ultralytics framework. Specifically, we focus on adversarial patches-carefully crafted patterns overlaid on an image to mislead object detectors into ignoring or misclassifying objects. We create effective naturalistic adversarial patches for recent versions of the Ultralytics YOLO models (YOLOv5, YOLOv8, YOLOv9, and YOLOv10) by upgrading an existing state-of-the-art patch generation approach designed for an earlier YOLOv4 model. We evaluate the attacks on the INRIA and on the MPII datasets across multiple YOLO models. The experimental results demonstrate high effectiveness in object detection evasion in recent models. The results suggest that larger models are more robust to adversarial attacks than their smaller counterparts. We run inference experiments on edge AI devices to highlight the response time improvement achieved by using smaller models, albeit at a higher risk of attack.

Keywords Adversarial attack · Adversarial example · Naturalistic patch · Machine learning · Deep learning · Object detection · Edge artificial intelligence

1 Introduction

Machine learning (ML) methods, including deep learning (DL), have shown remarkable performance in various domains such as computer vision, speech and facial recognition, autopilot and autonomous vehicles, the Internet of Things, and cybersecurity. Notwithstanding, the systems using these artificial intelligence (AI) methods are vulnerable to adversarial input perturbations designed to fool them either during training or testing, raising significant concerns. Adversarial machine learning (AML) focuses on the study of a category of attacks designed to degrade the accuracy and reliability of classifiers in performing their assigned tasks.

Common adversarial attacks are categorised as evasion [1], which alters inputs during the testing phase to produce incorrect outputs; poisoning [2], which corrupts training data to compromise model integrity; inversion [3], which uses model predictions to extract the training data; and model extraction [4], which uses a query access to the target model to obtain a new model whose performance is equivalent to the target model. These attacks often exploit the inherent vulnerabilities in ML and DL models by introducing carefully crafted perturbations that remain imperceptible to humans yet significantly impact model predictions. Specifically, adversarial examples-also referred to as adversarial patches

✉ J. Molleda
jmolleda@uniovi.es

D. L. Gala
danilopezgala@gmail.com

R. Usamentiaga
rusamentiaga@uniovi.es

1 Polytechnic School of Engineering, University of Oviedo, Campus of Gijon, Asturias, Spain

2 Department of Computer Science and Engineering, University of Oviedo, Campus of Gijon, Asturias, Spain

in some contexts-are small, intentional modifications applied to input data, causing the model to misclassify them with high confidence. Such perturbations can be generated through techniques like the Fast Gradient Sign Method (FGSM) [5], Projected Gradient Descent (PGD) [6], Carlini & Wagner (C&W) [7], Localised and Visible Adversarial Noise (LaVAN) [8], and One-Pixel [9]. FGSM, PGD, and C&W prioritise imperceptibility and optimization efficiency in digital domains, typically requiring full access to model gradients and the input at inference time. While highly effective in controlled, white-box settings, such methods are constrained by their dependence on real-time, per-image perturbation computation, limiting their applicability in scenarios where adversarial inputs must be precomputed, reused, or deployed in the physical world. The LaVAN attack introduces localised adversarial noise confined to a small image region, targeting object detectors without altering the entire image; however, it lacks naturalism and often produces conspicuous artifacts. Nonetheless, naturalistic adversarial patches, such as those explored in this work, emphasise practicality, realism, and cross-model transferability.

Traditional cloud computing, where large AI models typically run, faces significant challenges, such as high latency, bandwidth consumption, and potential privacy issues. Although large models have greater knowledge capacity than small models, they can be computationally expensive to evaluate, typically scaling quadratically with the number of parameters. Small models are typically generated by model distillation, the process of transferring knowledge from a large model to a smaller one. Small models offer several key advantages: *i*) Efficiency: they use fewer parameters, making them more cost-effective to operate; *ii*) Flexibility: their simplicity makes them easier to replicate; *iii*) Reliability: faster deployment times and the ability to run more replicas at the same cost improve system reliability; *iv*) Inference on CPU: if small enough, these models can perform inference using CPU SIMD instructions (e.g, AVX-512, Neon/SVE), which is significantly cheaper than using accelerators; *v*) Predictability: lower inference latency and reduced tail distribution make system performance more predictable; *vi*) Throughput: the ability to run more replicas increases parallelism and overall throughput; and *vii*) on-device: smaller models are easier to port for on-device inference, which is not only a growing trend but also necessary for specific use cases such as personal Retrieval Augmented Generation (RAG) and autonomous agents.

With the rapid growth of data and the Internet of Everything, edge computing, which processes data closer to its source, offers a promising solution by relieving the burden on data centres and reducing latency [10]. Many companies, such as large cloud providers, telcos or regional data centre providers, are starting to offer edge resources as a service (including compute, storage and networking) close to end users [11]. As the AI industry embraces the edge as part of its infrastructure, the security of AI models is a critical issue. Edge devices must run smaller models with fewer resources to achieve the same latency as cloud-based models running on more powerful servers, making the former more vulnerable to adversarial attacks.

In this work, we aim to generate and evaluate the effectiveness of adversarial patches for both large and small object detection models that have not previously been subjected to such attacks, or that achieve a level of success not previously documented. Previous research has demonstrated various methods for creating adversarial patches against object recognition models, including misleading attention and classification techniques [12], trust region-based optimisation approaches [13], and generative adversarial network-based attacks under complex conditions [14]. However, these methods either focus on different attack strategies or target specific object detection models. In contrast, our approach builds on and extends the work of [15], which trained adversarial patches for YOLOv4, by adapting it for compatibility with the widely used Ultralytics framework[1]. This adaptation includes modifying the framework to support backpropagation of gradients for adversarial patches, allowing researchers to train adversarial patches for object recognition models using diverse optimisation techniques.

The main contributions of this work can be summarised as follows:

- We created effective naturalistic adversarial patches, i.e. patches that resemble real objects, for recent versions of the Ultralytics YOLO models (YOLOv5, YOLOv8, YOLOv9 and YOLOv10) by upgrading and customizing an existing patch generation approach designed for an older YOLOv4 model.
- We tailored the Ultralytics framework to enable the use of YOLO models as detectors in the training phase of naturalistic adversarial patch generation.
- We conducted extensive experiments to evaluate the effectiveness of the naturalistic adversarial patches, generated using different Ultralytics YOLO models in the training phase, across multiple YOLO models. The transferability of adversarial patches, defined as their ability to effectively deceive models other than the one used for their generation, was also evaluated.
- We performed model inference experiments on edge AI devices with the naturalistic patches generated in this work, highlighting the importance of research in adversarial machine learning for developing robust and secure models that balance security and performance, especially for edge devices.

[1] https://www.ultralytics.com/

The rest of the paper is structured as follows. Section 2 reviews prior research on the adversarial security of ML and DL. Section 3 details the approach we rely on to create naturalistic adversarial examples for the Ultralytics YOLO object detectors. Sections 4 and 5 present the experimental results and subsequent discussion, respectively. Finally, we draw the conclusions of this work in Section 6.

2 Related work

Thorough reviews of work on the adversarial security of machine learning, pattern recognition, and deep neural networks have been developed in recent years. In [16], a threat model framework for evaluating the security of machine learning algorithms is outlined, detailing the attacker's goals, knowledge, and capabilities. Defence strategies were categorised into reactive and proactive approaches. Reactive defences, such as adversarial training and data sanitisation, respond to identified threats, while proactive defences, such as robust model design and defensive distillation, aim to anticipate and mitigate potential attacks before deployment. In [17], the mechanisms by which adversarial attacks are executed during both the training and testing phases is explored, and emphasised the importance of developing more robust models capable of withstanding adversarial manipulation. They also emphasised efficient privacy protection methods, such as differential privacy and homomorphic encryption, to protect sensitive data and model parameters.

In the cybersecurity domain, [18] examines the impact of AML, with a particular focus on on intrusion detection systems (IDS) and malware detection scenarios. The use of generative adversarial networks (GANs) and genetic algorithms to generate adversarial malware examples were identified. Despite the focus on attack strategies, defence mechanisms such as adversarial training, gradient masking, defensive distillation, and feature squeezing were also discussed. In [19], an in-depth examination of the security challenges and the significant security risks faced by connected and autonomous vehicles (CAVs) is provided, which rely heavily on ML and DL, due to AML. These vehicles are also vulnerable to attacks from the physical world, where adversarial examples may include altering traffic signs and generating adversarial perturbations that evade detection by state-of-the-art object detection models such as YOLO and Mask R-CNN. AML is surveyed in [20] with a focus on defending against adversarial attacks in image classification. DL algorithms, in particular convolutional neural networks (CNNs), have achieved state-of-the-art performance in various image classification tasks and are increasingly used in safety-critical applications such as biometric recognition systems and medical devices. These algorithms are vulnerable to adversarial examples, such as images with subtle perturbations, which can fool classifiers into making incorrect predictions with high confidence. The attacks have been categorised based on perturbation scope, visibility, and measurement, highlighting the different types of adversarial images such as individual-scoped and universal-scoped perturbations.

More recently, a comprehensive review of existing defence mechanisms against adversarial attacks has been conducted [21]. This review categorizes defence strategies into mitigation and detection methods, and evaluates their effectiveness in domains such as audio, cybersecurity, natural language processing (NLP), and computer vision. On the one hand, mitigation strategies, including adversarial training, defensive distillation, ensemble methods, and preprocessing techniques, aim to improve the robustness of ML models by incorporating adversarial examples during training, transferring knowledge from robust models, and transforming input features. Adversarial training, widely used in computer vision and explored in other domains, involves training models with adversarial examples to improve robustness. Defensive distillation and ensemble methods improve model robustness by exploiting knowledge transfer and combining multiple models, respectively. On the other hand, detection strategies focus on identifying and rejecting adversarial examples through supervised and semi-supervised learning, distance-based methods, feature squeezing, and statistical approaches. These strategies enhance the ability of ML models to detect anomalies and maintain integrity in the presence of adversarial perturbations.

In the domain of object detection, several works have been carried out for early versions of YOLO models. A robust physical adversarial sticker generation method is proposed in [22], based on an extended Expectation Over Transformation (EOT) for tiny-YOLO and the experiments were validated on random images from ImageNet. A serious adversarial vulnerability in YOLO detectors is identified in [23] by evaluating the impact of adversarial attacks sourced from the multiple task losses (i.e., objectness, localisation, and classification) and designed an effective attack for YOLOv4. The authors state that this attack is likely to be applicable to other detectors, but no experiments are provided to support this claim. The vulnerability of DL-based object detectors to adversarial attacks is analysed in [24], particularly in complex occlusion scenarios. A two-stage method based on Generative Adversarial Networks (GANs), called TS-GAN, was proposed. In the first stage, a generator is trained using an occlusion rule to simulate different occlusion scenarios, while in the second stage, latent variables are optimised to produce the most effective adversarial patches. The performance of these adversarial patches has been evaluated evaluated using the YOLOv2, YOLOv4tiny and Faster R-CNN object detectors. More recently, naturalistic adversarial patches have been generated in [25], resembling potted plants, specifically designed

for the YOLO object detectors. These patches, generated based on an entropy-boosted loss function, obscure the ability of the detectors to identify people in the scene. Evaluated on the INRIA person dataset, this method evades the pedestrian detection limited to the YOLOv2, YOLOv3, and YOLOv4 detectors. Although the authors state that this method could be applied to any object detection model where the probability values are calculated using Softmax, this is a theoretical assumption and it is not supported by experiments.

There are really few works related to the generation of adversarial examples for recent YOLO detectors, such as YOLOv8, YOLOv9 and YOLOv10. So far, the only one that achieves this goal with a limited improvement over previous attacks was recently developed in [26], by generating adversarial examples for object detectors considering both optimisation methods and data augmentation techniques. The authors exploit the similarities between adversarial sample generation and neural network training. The proposed data augmentation framework, together with the combination of weighted average gradient and momentum, further improves the attack success rate and transferability of adversarial examples. This proposal has been validated on Faster R-CNN, YOLOv3 and YOLOv8 using the COCO dataset.

3 Methods

3.1 Naturalistic Adversarial Patches

This work focuses on the generation of adversarial patches that resemble real objects in order to evade the detection of people on the latest Ultralytics YOLO object detector models. The adversarial patches are generated by optimizing the latent space of a pre-trained GAN rather than directly modifying pixel values. This approach leverages the natural image manifold learned by the GAN, thereby producing patches that appear realistic while fooling the detector.

The generation process begins with a GAN generator G that is pre-trained on the ImageNet [27] dataset. A latent vector $z \in \mathbb{R}^d$ is sampled from a standard normal distribution to produce an initial patch $P = G(z) \in \mathbb{R}^{H \times W \times 3}$. The patch is then iteratively refined by updating z via gradient descent in order to minimize the overall loss function defined as:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{det}} + \lambda_{\text{tv}} \mathcal{L}_{\text{tv}}, \tag{1}$$

where $\mathcal{L}_{\text{det}}$ is the detection loss and $\mathcal{L}_{\text{tv}}$ is a total variation loss that enforces spatial smoothness. In this work, the coefficient λ_{tv} is set to 0.1 to balance the smoothness constraint with the adversarial objective.

The detection loss is defined as:

$$\mathcal{L}_{\text{det}} = \frac{1}{N} \sum_{i=1}^{N} \max_j \left[D^j_{\text{obj}}(I'_i) \cdot D^j_{\text{cls}}(I'_i) \right], \tag{2}$$

where:

- N is the number of images in the current batch.
- I'_i denotes the i-th image with the adversarial patch applied.
- The index j iterates over all detections produced by the object detector for image I'_i.
- $D^j_{\text{obj}}(I'_i)$ is the objectness score for the j-th detection, measuring the likelihood that the detected region contains an object.
- $D^j_{\text{cls}}(I'_i)$ is the classification score for the j-th detection, indicating the confidence that the detected object belongs to the target class.

The purpose of $\mathcal{L}_{\text{det}}$ is to reduce the highest detection confidence in each patched image. Minimizing the maximum product of the objectness and classification scores forces the detector to either fail to detect the target or misclassify it, thereby achieving the adversarial goal.

The total variation loss, which promotes smoothness in the patch, is given by:

$$\mathcal{L}_{\text{tv}} = \sum_{i,j} \sqrt{(P_{i+1,j} - P_{i,j})^2 + (P_{i,j+1} - P_{i,j})^2}. \tag{3}$$

Once the total loss is computed, gradient updates are performed on the latent vector z. To prevent drastic changes, an ℓ_∞ norm constraint is applied during the update. The update rule is defined as:

$$z^t = \kappa \left(z^{t-1} + \eta \nabla \mathcal{L}_{\text{total}} \right), \tag{4}$$

where η is the step size and $\nabla \mathcal{L}_{\text{total}}$ is the gradient of the total loss with respect to z. The function $\kappa(\cdot)$ clips each component of z to lie within $[-\tau, \tau]$, as defined below:

$$\kappa(z) = \{z_i \mid z_i \leftarrow \min\big(\max(z_i, -\tau), \tau\big),\ z_i \sim z\}. \tag{5}$$

To further clarify the methodology, Algorithm 1 presents the complete workflow for generating naturalistic adversarial patches.

Two datasets are used in this work: *i*) the INRIA person dataset [28], which contains images of people captured by closed circuit television (CCTV) cameras; and *ii*) the MPII human pose dataset [29], which contains images of people in every day human activities. The optimisation process uses the Adam optimiser with the settings shown in Table 1.

Table 1 Configuration settings for the optimisation process in the proposed experiments

Parameter	Configuration settings
Optimiser	Adam optimiser with a learning rate of 0.01 while we experiment with different learning rate parameters in the following experiments, $\beta_1 = 0.5$, and $\beta_2 = 0.999$. The learning rate is adjusted downwards if the change in loss is below 1×10^{-4} for 50 epochs continuously.
Batch size	8 to 32, depending on the hardware resources available.
Epochs	400 to 1000, depending on the experiment.
Generator	BigGAN, pre-trained on ImageNet-1k with a resolution of 128×128, to generate class-specific adversarial patches.
Latent vector	For BigGAN, a latent vector dimension, d, of 120 is used.
Object detector	Ultralytics YOLOv5, YOLOv8, and YOLOv9 models.

Algorithm 1 Adversarial Patch Generation

Input: Pre-trained GAN G, object detector, image batch $\{I_i\}_{i=1}^{N}$, step size η, clip threshold τ, smoothness weight λ_{tv}, iterations T
Initialize: Sample $z \sim \mathcal{N}(0, I)$ and compute initial patch $P = G(z)$.
for $t = 1$ **to** T **do**
  for each image I_i in the batch **do**
    Apply patch P to I_i to obtain patched image I_i'.
    Run object detector on I_i' to compute detection loss $L_{\text{det}}(I_i')$.
  end for
  $\mathcal{L}_{\text{det}} = \frac{1}{N} \sum_{i=1}^{N} L_{\text{det}}(I_i')$.
  $\mathcal{L}_{\text{tv}}$ on P.
  Form total loss: $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{det}} + \lambda_{\text{tv}} \mathcal{L}_{\text{tv}}$.
  Update latent vector: $z \leftarrow \kappa\left(z + \eta \nabla \mathcal{L}_{\text{total}}\right)$.
  Update patch: $P = G(z)$.
end for
Output: Final adversarial patch P.

The naturalistic adversarial patch generation process developed in this work is illustrated in Fig. 1. First, objects in the original image are detected using a preselected object detection model from the Ultralytics framework. Next, the naturalistic looking adversarial patch is generated by the GAN. Then, a transformation process is applied to the original image to overlay the adversarial patch centred on the bounding boxes of the objects detected in the first step, yielding the patched image. Finally, the object detection model is re-run on the patched image to obtain the detection results. These results are used to compute the detection loss function, denoted by $\mathcal{L}_{\text{det}}$. This is followed by the addition of the total variation loss function $\mathcal{L}_{\text{tv}}$ to give the total loss, $\mathcal{L}_{\text{total}}$. The total loss is used to update the latent vector, z, of the GAN. This process is repeated until the adversarial patch is generated. The stopping criterion used in the patch generation to ensure that the model has learned effectively without overfitting or underfitting is a fixed number of epochs. ReduceLROnPlateau with a patience of 50 epochs is employed in all experiments. Early stopping is not used, so training is always conducted for the predetermined number of epochs regardless of the learning rate.

3.2 Performance evaluation metrics

In this work, we use the mean average precision (mAP), a common evaluation metric in object detection tasks to assess the performance of models such as YOLO [30]. This metric summarises the performance of the model across all classes and detection thresholds. The computation of mAP involves several fundamental metrics:

- Intersection over Union (IoU): provides a measure of the overlap between the predicted bounding box (B_p) and the ground truth bounding box (B_g) of a target object, calculated using Eq. 6.

$$IoU(B_g, B_p) = \frac{\text{area}(B_g \cap B_p)}{\text{area}(B_g \cup B_p)}. \tag{6}$$

- Precision and Recall: Precision (P) indicates the correctness of the detected objects, while Recall (R) measures the ability of the model to detect relevant objects. They are calculated using Eq. 7 and Eq. 8, respectively, where S is the number of true positive detections, N is the total number of predicted detections, and G is the total number of ground truth instances.

$$P = \frac{\sum_{n=1}^{S} \text{TP}_n}{\sum_{n=1}^{S} \text{TP}_n + \sum_{n=1}^{N-S} \text{FP}_n} \tag{7}$$

$$R = \frac{\sum_{n=1}^{S} \text{TP}_n}{\sum_{n=1}^{S} \text{TP}_n + \sum_{n=1}^{G-S} \text{FN}_n} \tag{8}$$

  The Precision-Recall curve is generated by varying the detection confidence threshold and plotting the precision against recall at each threshold. This allows for a comprehensive evaluation of the model's performance across different levels of detection confidence.
- Average Precision (AP): for each class, the area under the Precision-Recall curve represents a single scalar value that summarises the precision of the model across differ-

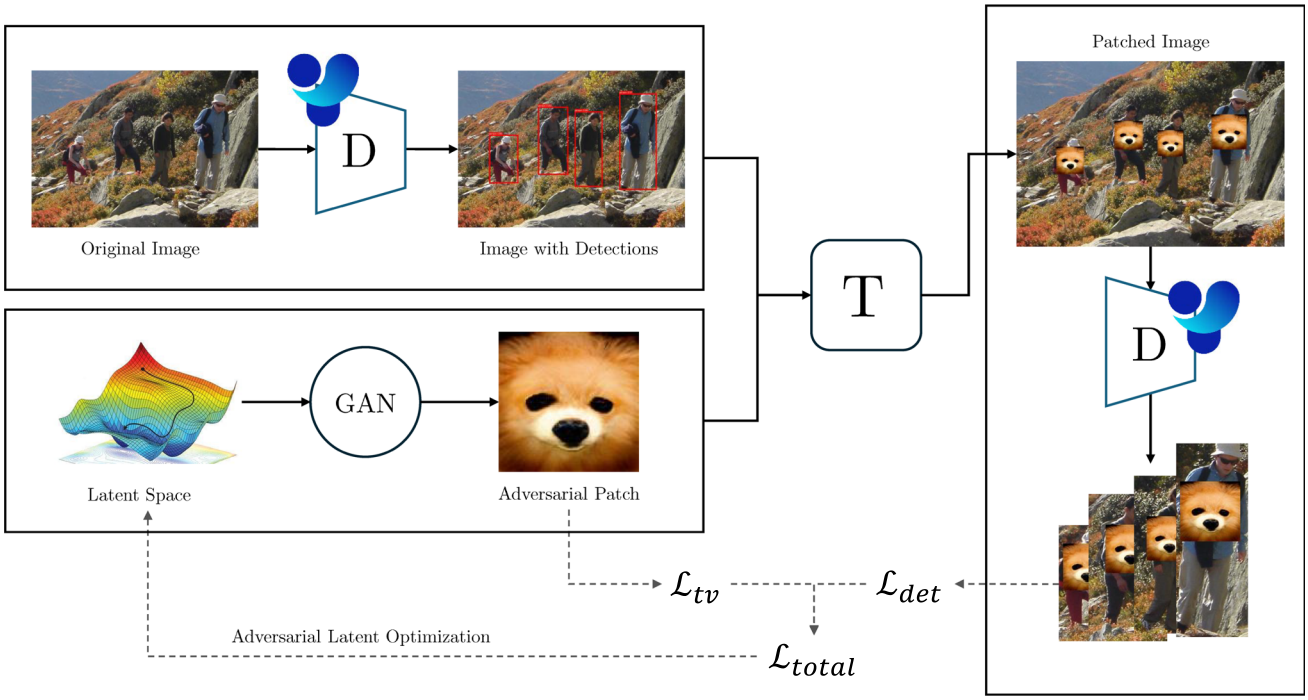

Fig. 1 Adversarial patch generation process

ent recall levels. It is typically calculated for each object class and different confidence thresholds using numerical integration methods such as the 11-point interpolation or more dense sampling of recall values. AP can also be approximated as detailed in Eq. 9, where P_n and R_n are the precision and recall at the n th threshold. AP captures the trade-off between precision and recall, demonstrating the ability of the model to maintain high precision while maximising recall.

$$AP = \sum_{n=1}^{N} (R_n - R_{n-1}) P_n \tag{9}$$

Therefore, mAP aggregates the AP over all classes and possibly over a range of IoU thresholds, providing a complete metric of model performance. For ℓ object classes and IoU threshold μ, mAP is defined as Eq. 10. When considering a range of IoU thresholds $(\mu_1, \ldots, \mu_J)$, mAP over this range increases the robustness of the metric, as shown in Eq. 11.

$$mAP(\mu) = \frac{1}{\ell} \sum_{i=1}^{\ell} AP(C_i, \mu). \tag{10}$$

$$mAP(\mu_1, \ldots, \mu_J) = \frac{1}{J} \sum_{j=1}^{J} mAP(\mu_j). \tag{11}$$

As mentioned above, the datasets used in this work are the INRIA person dataset and the MPII human pose dataset. Since these dataset contain only a single class, *Person*, the mAP is equivalent to the AP for the *Person* class.

4 Experimental results

We have conducted an experimental design to evaluate the effectiveness of adversarial patches across the latest YOLO models generated using the approach described above. The results of these experiments are presented in this section.

4.1 Hardware platforms

The experiments conducted in this work were performed on three hardware platforms, as shown in Table 2. Both the server and workstation platforms were used for both adversarial patch generation and person detection through model inference, while the edge AI device was used exclusively for model inference. To run the experiments on these systems, several cross-platform Docker containers were crafted to ensure the reproducibility of the results and ease of deployment across different environments.

Table 2 Specifications of the platforms used in the training and inference phases

	Server	Workstation	Edge AI device
GPU	NVIDIA GRID A100D-2-20C	NVIDIA GeForce RTX 2080 Ti	NVIDIA GM20B Maxwell 2.0
	2 compute pipelines, 20 GB HBM2e	11 GB GDDR6	4 GB LPDDR4 (shared with CPU)
CPU	AMD EPYC 7413	Intel Core i7-9700K	ARM Cortex-A57
	24-core processor, 48 threads	8-core processor, 8 threads	4-core processor
	2.65 GHz, turbo boost up to 3.6 GHz	3.6 GHz, turbo boost up to 4.9 GHz	1.43 GHz

4.2 Implementation details

In order to use the Ultralytics framework as a detector in the optimisation phase of patch generation, both the framework and the adversarial patch generation method have been upgraded. The modifications to both software packages are available via public GitHub repositories.[2,3] Firstly, it was necessary to implement the training, evaluation, and loss function calculation processes. Second, the gradients of the predictions of the YOLO models had to be propagated through the loss functions to update the adversarial patches.

(a) (b) (c)

Fig. 2 Comparison of object detection for the *Person* class with different patch scales: (a) no patch, $Confidence\ Score(Person) = 0.91$; (b) patch scale: 0.15, $Confidence\ Score(Person) = 0.82$; (c) patch scale: 0.20, $Confidence\ Score(Person) = 0.72$

4.3 Evaluation

The adversarial patches generated in these experiments were initially evaluated using the INRIA Person dataset. Subsequently, the most effective patches and method configurations were assessed on the MPII Human Pose dataset to better demonstrate the generalizability of the adversarial attacks. Together, these evaluations also served to assess the implications for edge AI security.

Initial exploratory experiments revealed that images patched with smaller-scale patches were less effective at evading detection. In contrast, larger-scale patches were more successful but were also more conspicuous and less realistic. Consequently, the primary evaluation was conducted using a patch scale of 0.20, with additional scales tested to assess the influence of patch size on detection performance. Fig. 2 illustrates the effect of path size on the detection of instances of the *Person* class in an image.

For all images in the dataset, the performance of the models without the adversarial patches is consistently 100%, as the IoU is 1. This is because the original detections are used as ground truth for evaluation. As a baseline, the effectiveness of white, grey and random patches is compared with the adversarial patches generated in [15] for YOLOv4 (*patch-v4*), YOLOv4t (*patch-v4t*) and Faster R-CNN. These baseline patches are generated following a standard approach where a white patch is represented by all pixel values set to 1, a grey patch is initialised with pixel values set to 0.5, and a random patch consists of pixel values drawn from a uniform distribution between 0 and 1. This methodology is consistent with common practices in adversarial robustness evaluation. In addition, the effectiveness of these baseline patches is compared with two well-known adversarial patch generation methods proposed in [31], based on the objectness score (*advyolo* and *advyolo-up*). The performance of the baseline patches is shown in Table 3, Table 4 and Table 5 for patch scales 0.15, 0.20 and 0.25 respectively. These and other tables use the colour legend shown in Fig. 3.

The naturalistic adversarial patches are generated using the method described in this work for the YOLOv5n, YOLOv5s, YOLOv5m, YOLOv8n, YOLOv9t, YOLOv9s, YOLOv9m models and evaluated on the same models and the latest models such as YOLOv10n, YOLOv10s, and YOLOv10m. This evaluation also assesses patch transferability across models. Table 6 presents the results of the experimental design, which includes the five key parameters required to generate adversarial patches: *i)* the model used as the detector, *ii)* the class of the patch, *iii)* the number of epochs, *iv)* the weight of the total variation loss, and *v)* the learning rate. It can be observed that while smaller models are more vulnerable to attack, larger models produce more effective adversarial patches when used as detectors. The *Peacock* and *Dog* classes were predominantly used in the experiments due to their observed effectiveness in generating patches, with the *Penguin* class also used in one experiment. The number of epochs was generally set to 1000 for most experiments, as the loss typically converges within this range.

[2] https://github.com/Bimo99B9/ultralytics

[3] https://github.com/Bimo99B9/NaturalisticAdversarialPatches

Table 3 mAP results for baseline patches with a scale of 0.15 on the INRIA person dataset: *patch-v4* and *patch-v4t* computed using [15]; *faster R-CNN*, *advyolo*, and *advyolo-up* computed using [31]

	Patch (scale = 0.15)							
Model	*none*	*white*	*random*	*patch-v4*	*patch-v4t*	*faster R-CNN*	*advyolo*	*advyolo-up*
v5n	100.00	77.37	85.41	84.84	76.91	83.62	76.22	78.76
v5s	100.00	80.87	87.29	83.07	80.31	81.17	72.44	71.32
v5m	100.00	83.68	85.22	82.44	82.20	83.86	74.85	77.46
v8n	100.00	79.47	85.11	84.67	79.86	81.25	70.86	75.85
v8s	100.00	81.89	87.74	84.81	81.12	84.35	75.64	72.26
v8m	100.00	83.57	84.55	80.51	81.58	83.18	72.56	78.47
v9t	100.00	80.76	86.61	84.30	82.71	84.47	81.35	78.56
v9s	100.00	82.74	88.06	84.60	81.76	84.83	78.67	79.27
v9m	100.00	84.32	84.98	80.68	83.86	81.35	72.18	78.39
v10n	100.00	83.18	87.58	86.35	75.19	77.59	76.23	76.74
v10s	100.00	84.22	87.66	83.97	80.39	81.36	75.37	76.51
v10m	100.00	84.26	86.71	82.96	79.51	81.43	72.46	77.29

Table 4 mAP results for baseline patches with a scale of 0.20 on the INRIA person dataset: *patch-v4* and *patch-v4t* computed using [15]; *faster R-CNN*, *advyolo*, and *advyolo-up* computed using [31]

	Patch (scale = 0.20)							
Model	*none*	*white*	*random*	*patch-v4*	*patch-v4t*	*faster R-CNN*	*advyolo*	*advyolo-up*
v5n	100.00	62.10	77.08	73.82	50.65	61.21	58.42	61.85
v5s	100.00	68.11	78.14	71.23	57.53	56.49	51.19	42.69
v5m	100.00	76.23	79.19	72.40	68.89	70.22	60.80	62.81
v8n	100.00	67.94	77.13	75.31	56.48	63.34	52.59	51.22
v8s	100.00	74.81	79.37	72.97	58.36	58.74	53.75	45.10
v8m	100.00	76.77	78.34	70.87	69.54	68.26	51.15	63.22
v9t	100.00	72.53	79.32	78.21	67.06	71.27	66.17	62.46
v9s	100.00	75.13	80.28	73.05	61.96	68.29	62.45	65.47
v9m	100.00	77.42	77.97	70.02	70.79	67.41	51.87	68.16
v10n	100.00	72.93	81.60	75.56	49.88	49.15	59.08	57.09
v10s	100.00	76.80	79.49	69.00	59.06	59.37	51.73	55.83
v10m	100.00	76.62	79.86	73.26	64.36	70.01	53.94	59.25

Table 5 mAP results for baseline patches with a scale of 0.25 on the INRIA person dataset: *patch-v4* and *patch-v4t* computed using [15]; *faster R-CNN*, *advyolo*, and *advyolo-up* computed using [31]

	Patch (scale = 0.25)							
Model	*none*	*white*	*random*	*patch-v4*	*patch-v4t*	*faster R-CNN*	*advyolo*	*advyolo-up*
v5n	100.00	26.18	56.42	45.89	16.60	27.09	32.68	36.99
v5s	100.00	39.53	59.35	36.06	21.49	21.51	25.71	21.12
v5m	100.00	62.05	65.68	42.17	37.12	37.21	40.45	52.99
v8n	100.00	36.86	66.33	44.49	24.77	33.63	23.40	28.56
v8s	100.00	61.14	66.69	42.05	25.84	23.57	26.13	26.09
v8m	100.00	65.26	64.26	47.39	45.44	38.84	30.05	52.30
v9t	100.00	60.12	69.65	65.33	33.13	46.60	39.97	43.73
v9s	100.00	64.67	71.55	45.89	31.27	42.55	34.44	43.39
v9m	100.00	64.48	67.75	44.20	49.32	34.25	31.59	59.08
v10n	100.00	54.66	72.25	48.64	14.94	16.24	28.62	33.97
v10s	100.00	66.98	72.09	35.16	29.05	26.87	26.86	37.53
v10m	100.00	67.99	72.13	54.08	43.08	43.09	33.50	48.09

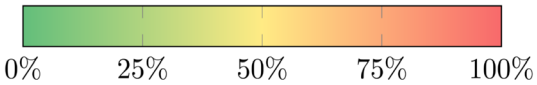

Fig. 3 Colour legend for mAP visualization used in this work

However, in some experiments fewer epochs were used to investigate potential overfitting and underfitting issues. The total variation loss weight ranged from 0 to 0.1, with optimal results achieved at a weight of 0.1. The learning rate was adjusted between 0.005 and 0.02, using the Adam optimiser with betas of 0.9 and 0.999 across all experiments. Given the effectiveness of larger models, the YOLOv5m model was predominantly used as the detector in most experiments. In addition, variations in the learning rate and total variation loss weight were explored to identify the most effective adversarial patches.

Fig. 4 illustrates the error between the predicted outputs and the ground truth in the training data showing the moving average of the detection loss function over 1000 epochs for the generation of three adversarial patches using YOLOv5m as the model in the training phase: *patch17* (learning rate 0.005), *patch25* (learning rate 0.001), and *patch27* (learning rate 0.01). The shaded areas around each line represent the variability in detection loss. As can be seen, different learning rates have a significant effect on the convergence and stability of the detection loss during the training of adversarial patches, and thus produce slightly different adversarial patches even though they belong to the same class. On the other hand, Fig. 5 shows some example generated patches belonging to the classes *Peacock* and *Dog*.

Table 7 shows the mean average precision (mAP) for each adversarial patch (rows) evaluated on the INRIA person dataset across different YOLO model versions (columns) at a fixed patch scale of 0.20. By reading each row from left to right, one can see how effectively each patch disrupts person detection for smaller (e.g., YOLOv5n, YOLOv8n) and larger (e.g., YOLOv5m, YOLOv10m) models. In general, smaller models are more vulnerable and often have lower mAP values (in the 40–60% range), while medium models tend to have higher mAP (often above 60–70%). Some patches, such as *patch17*, *patch26*, *patch27* and *patch38*, repeatedly produce more pronounced drops in mAP across multiple YOLO variants, indicating their strong adversarial effect. However, the same patch can have different effects on different YOLO variants, suggesting partial transferability: while certain adversarial features carry over between architectures, the unique design of each model still influences how it responds to the patch. For example, *patch27* lowers YOLOv5n detection mAP to around 49%, but YOLOv5m mAP remains at around 56%, suggesting that YOLOv5m larger capacity provides some resilience.

For all the experiments run, the performance of the baseline patches, such as the white patch, the grey patch and the random patch, is used as a benchmark. The comparison of the baseline patches with the four best patches generated in the experiments, on the INRIA person dataset, is shown in Table 8 for a patch scale of 0.20. To further assess the effect of patch scale, Table 9 shows the results of the same comparison for a patch scale of 0.22. As can be seen, increasing the patch size by 10 % significantly improves the effectiveness of the attacks. The random patch is the least affected by this increase, with an average improvement of 4.5 %, while the *patch27* patch benefits the most, with an average improvement of 27.3 %. In addition to the previous evaluations, we run the experiments on the MPII human pose dataset. The comparison of the baseline patches with the four best patches generated in the experiments is shown in Table 10 for a patch scale of 0.20.

5 Discussion

In the above experiments, we have shown that different naturalistic adversarial patches can be generated to effectively evade the detection of pedestrians in images by the latest Ultralytics YOLO models. The four most effective patches on a scale of 0.20, denoted as *patch17*, *patch26*, *patch27* and *patch38*, showed an average mAP of 55.72%, 55.09%, 51.43% and 51.47%, respectively, across all models evaluated on the INRIA person dataset. In comparison to the baseline presented in Table 4, the experiment designated as *patch27* showed a 52.22% improvement over the random baseline for the YOLOv10s model, and the patch *patch38*, showed a 51.92% improvement for the YOLOv10n model. The improvement achieved in mAP by these four adversarial patches over the random patch and to the *advyolo* method [31] is shown in Table 11. This comparison shows that in all cases the patches generated by the method developed in this work exceed the evasion achieved by a random patch. Moreover, in two thirds of the cases (32 out of 48), the patches generated by this work are more effective (up to 37.1 % in the best case) than those generated in [31], the best performing of the baselines. The patches generated in this work are more effective on tiny and small models, except for YOLOv8s.

The results highlight the generalizability and robustness of the adversarial patches across different datasets. While baseline patches (white, grey, random) and prior methods such as Faster R-CNN and *advyolo* showed moderate effectiveness, the patches generated in this work consistently achieved lower mAP values, indicating stronger attack performance. Notably, the performance degradation was even more pronounced on the MPII dataset, as shown in Table 10, where several patches reduced mAP below 20% for certain models. As can be seen, all YOLO models tested remained vulnerable

Table 6 Summary of the experimental design followed to generate the naturalistic adversarial patches

Patch	Model	Patch class	Epochs	Weight $\mathcal{L}_{\mathbf{tv}}$	Learning rate
patch01	v8n	Peacock (84)	1000	0	0.005
patch02	v8n	Peacock (84)	643	0.1	0.005
patch03	v8n	Dog (259)	800	0.05	0.005
patch04	v5n	Peacock (84)	1000	0	0.005
patch05	v5n	Peacock (84)	1000	0.1	0.005
patch06	v5n	Peacock (84)	300	0.05	0.005
patch07	v5n	Peacock (84)	300	0.05	0.005
patch08	v5n	Peacock (84)	1000	0.1	0.005
patch09	v5n	Peacock (84)	1000	0.1	0.005
patch10	v5n	Peacock (84)	400	0.1	0.005
patch11	v5n	Peacock (84)	400	0.1	0.005
patch12	v5n	Peacock (84)	1000	0.1	0.005
patch13	v5n	Peacock (84)	363	0	0.005
patch14	v5n	Dog (259)	1000	0.1	0.005
patch15	v5n	Dog (259)	700	0.05	0.005
patch16	v5s	Dog (259)	1000	0.1	0.005
patch17	v5m	Dog (259)	1000	0.1	0.005
patch18	v5m	Dog (259)	1000	0.05	0.02
patch19	v5m	Peacock (84)	800	0.05	0.03
patch20	v5l	Peacock (84)	350	0.05	0.005
patch21	v5m	Dog (259)	1000	0	0.0025
patch22	v5s	Chimp (367)	500	0	0.0025
patch23	v5m	Dog (259)	1000	0.1	0.005
patch24	v5m	Dog (259)	600	0.05	0.04
patch25	v5m	Dog (259)	1000	0.1	0.001
patch26	v5m	Dog (259)	1000	0.05	0.01
patch27	v5m	Dog (259)	1000	0.1	0.01
patch28	*v9t*	Dog (259)	1000	0.1	0.01
patch29	*v9s*	Dog (259)	1000	0.1	0.01
patch30	*v9m*	Dog (259)	1000	0.1	0.0025
patch31	*v9m*	Dog (259)	1000	0.1	0.005
patch32	*v9m*	Dog (259)	1000	0.1	0.01
patch33	*v9m*	Dog (259)	1000	0.05	0.01
patch34	*v9m*	Peacock (84)	1000	0.1	0.01
patch35	*v9m*	Dog (259)	1000	0	0.01
patch36	*v9m*	Dog (259)	1000	0.1	0.015
patch37	v5m	Dog (259)	1000	0.08	0.0085
patch38	v5m	Dog (259)	1000	0	0.01
patch39	v5m	Dog (259)	1000	0.05	0.01
patch40	v5m	Dog (259)	1000	0	0.015
patch41	v5m	Dog (259)	1000	0.05	0.02
patch42	v5m	Dog (259)	1000	0.1	0.001
patch43	v5m	Dog (259)	1000	0.1	0.0005

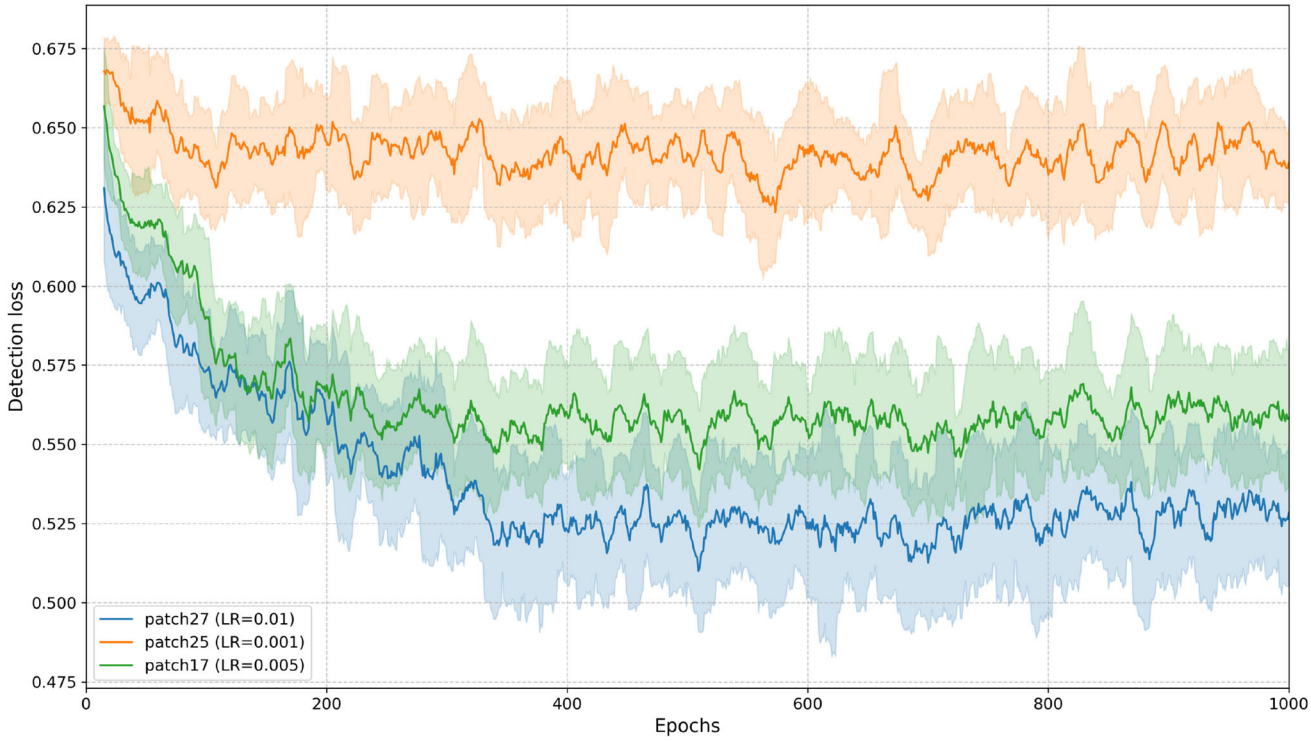

Fig. 4 Detection loss while training three adversarial patches: *patch17*, *patch25*, and *patch27*

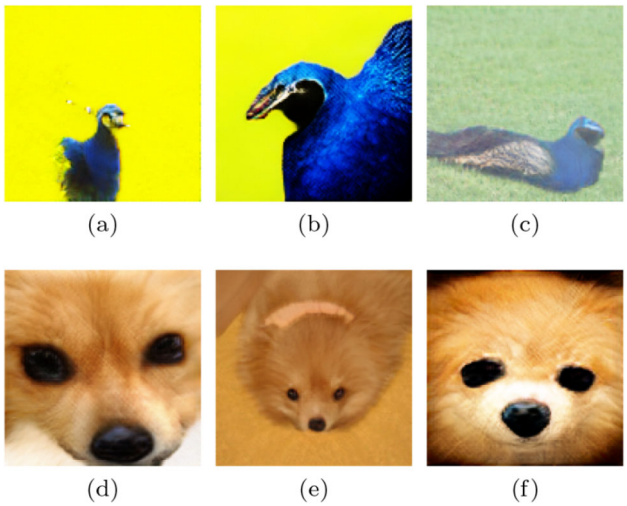

Fig. 5 Adversarial naturalistic patches: (a) *patch08*, *Peacock* class, generated with YOLOv5n; (b) *patch01*, *Peacock* class, generated with YOLOv8n; (c) *patch02*, *Peacock* class, generated with YOLOv8n; (d) *patch16*, *Dog* class, generated with YOLOv5s; (e) *patch43*, *Dog* class, generated with YOLOv5m; (f) *patch27*, *Dog* class, generated by YOLOv5m

to the attacks. These results confirm the transferability of the patches and emphasize their potential impact on real-world deployments.

The performance of these patches is significantly better than the baseline patches, demonstrating that the naturalistic adversarial patch generation method, when combined with the Ultralytics framework during training, is effective in generating patches capable of fooling the latest YOLO object detectors. It is also notable that the effectiveness of the naturalistic adversarial patches in YOLOv8 and YOLOv9 is less significant compared to earlier models such as YOLOv5. This observation may indicate that newer models are more robust to adversarial attacks. However, the same adversarial patches are more effective at evading YOLOv10, as shown in Table 7, indicating that newer models do not consistently show increase robustness to adversarial attacks.

An initial hypothesis for this variability could be related to the number of parameters in each model. Newer models tend to be more sophisticated and often require fewer parameters. For example, YOLOv8m has 25.9 million parameters, while YOLOv9m and YOLOv10m have 20.0 million and 15.4 million parameters, respectively. However, this hypothesis is challenged by a counterexample involving YOLOv9s and YOLOv10s, where YOLOv9s uses 7.1 million parameters and YOLOv10s uses 7.2 million, yet the adversarial attack is more effective on YOLOv10s than on YOLOv9s. This suggests that factors related to the model architecture may contribute to the susceptibility to adversarial attacks. For example, the YOLOv9 model architecture is based on vision transformers, while YOLOv10 is based on a more refined and complex model with multiple sequential convolutional layers [32].

Table 7 mAP of the most representative adversarial patches generated in this work across the attacked models, with a scale 0.20, on the INRIA person dataset

	Model											
Patch	**v5n**	**v5s**	**v5m**	**v8n**	**v8s**	**v8m**	**v9t**	**v9s**	**v9m**	**v10n**	**v10s**	**v10m**
patch01	74,69	66,67	71,90	70,09	74,34	70,36	78,41	75,80	74,04	74,10	75,74	74,41
patch02	66,36	69,14	74,02	66,57	72,41	71,49	70,47	71,65	71,89	69,32	72,08	73,70
patch03	68,07	69,02	75,33	68,19	73,52	73,09	71,18	73,58	74,81	72,81	75,19	73,90
patch04	73,74	69,10	73,19	72,32	72,01	70,42	76,78	74,97	74,41	75,72	75,38	75,38
patch05	60,28	62,42	74,82	64,15	69,83	74,08	70,78	73,17	72,23	68,41	72,36	73,74
patch06	61,99	65,22	73,47	61,15	71,00	74,54	69,00	73,77	75,48	68,06	72,91	75,58
patch07	56,62	60,92	73,84	61,83	69,69	76,58	69,74	74,53	75,00	70,13	73,94	74,99
patch08	58,82	62,28	72,42	62,33	67,95	70,98	69,38	72,68	73,00	66,22	69,97	72,06
patch09	59,80	57,05	71,93	58,72	65,81	72,63	70,02	70,49	71,32	67,73	68,67	73,91
patch10	57,26	66,36	74,78	60,10	71,85	77,43	70,07	74,36	77,43	69,01	73,25	75,15
patch11	58,18	58,25	69,54	59,81	64,15	70,46	68,17	70,01	70,30	65,79	67,54	71,52
patch12	64,24	62,39	71,70	62,06	67,20	72,42	69,06	73,63	72,07	69,26	69,83	74,26
patch13	76,16	70,04	73,26	74,69	72,01	69,87	77,45	76,93	74,10	77,40	75,94	75,94
patch14	60,38	65,58	74,80	70,52	71,94	74,66	70,28	76,78	74,53	64,04	70,80	72,34
patch15	64,62	54,93	72,54	65,28	69,88	72,65	72,17	72,32	70,28	68,40	68,09	71,18
patch16	65,05	46,92	70,06	63,16	68,85	70,00	76,30	70,62	65,24	61,10	56,90	66,40
patch17	48,51	47,54	61,21	52,08	60,92	65,74	65,90	62,18	67,01	46,19	43,36	57,82
patch18	60,74	51,32	63,07	61,20	67,34	67,85	72,20	66,95	65,31	53,37	57,91	63,46
patch19	63,11	47,38	62,99	58,28	62,53	69,22	69,90	70,05	68,64	59,82	54,49	64,51
patch20	74,95	75,61	74,74	71,38	74,33	75,32	76,08	75,51	76,57	74,55	74,41	73,17
patch21	54,12	49,29	61,87	54,52	62,53	65,49	67,22	63,46	66,59	48,18	44,92	56,90
patch22	68,84	66,54	71,66	70,80	70,80	69,74	72,17	72,72	71,77	74,81	70,95	70,90
patch23	51,91	47,54	63,22	58,82	63,95	66,10	65,34	65,46	69,79	49,18	44,37	60,01
patch24	70,66	66,08	72,71	68,98	73,87	71,49	71,26	74,31	73,02	70,58	70,16	67,23
patch25	73,70	69,61	73,99	74,15	72,79	72,20	75,27	75,02	72,69	73,88	74,05	74,22
patch26	52,01	50,81	58,95	53,73	59,00	63,37	63,79	59,36	65,25	48,24	41,16	55,10
patch27	49,42	47,70	56,04	52,46	54,95	61,26	61,81	53,55	61,52	46,17	38,23	51,98
patch28	64,00	63,86	70,97	64,34	70,27	73,37	71,24	70,97	70,17	67,62	69,31	70,77
patch29	50,77	50,54	67,74	55,06	64,41	68,41	65,08	61,85	66,45	61,24	64,58	68,77
patch30	67,50	55,12	70,23	68,76	64,81	69,75	76,45	65,30	65,09	62,27	61,55	68,86
patch31	77,38	67,60	69,17	74,89	72,17	65,34	81,89	78,05	64,18	77,50	75,40	72,10
patch32	51,42	44,06	68,22	56,30	54,82	68,11	65,30	60,33	57,88	44,86	48,89	61,94
patch33	67,50	59,26	58,07	63,30	65,69	64,52	72,62	73,97	61,46	69,86	58,39	64,39
patch34	66,31	61,47	73,67	63,96	67,20	70,15	73,39	73,51	71,80	72,17	66,16	74,85
patch35	76,93	61,24	65,61	74,72	70,22	68,50	81,65	73,70	66,37	76,56	69,90	68,65
patch36	72,72	66,28	73,11	67,39	75,19	71,91	72,36	72,79	72,65	74,19	77,45	72,05
patch37	49,52	45,98	61,08	51,70	60,20	64,75	66,33	61,73	66,99	45,66	41,78	58,11
patch38	36,76	45,99	59,52	48,31	58,13	65,50	60,37	57,85	65,63	39,20	39,94	48,65
patch39	75,39	70,57	68,87	72,23	76,04	71,56	79,35	76,06	73,63	77,31	68,30	66,90
patch40	50,78	47,77	57,85	57,36	65,33	65,47	62,85	60,87	61,44	48,96	49,61	55,58
patch41	44,39	41,35	63,24	53,04	58,83	65,19	60,74	57,38	64,68	42,30	43,75	59,34
patch42	73,44	69,63	73,78	73,65	72,91	72,30	75,28	75,14	72,79	73,40	74,03	74,39
patch43	73,38	72,65	74,02	75,05	73,62	74,19	75,15	75,81	73,31	77,50	75,03	74,47

Furthermore, the results consistently show that smaller models (YOLOv5n, YOLOv8n, YOLOv9n, and YOLOv10n) exhibit greater vulnerability to adversarial patches compared to their larger counterparts (YOLOv5m, YOLOv8m, YOLOv9m, and YOLOv10m), suggesting that lightweight architectures commonly deployed in edge AI systems may be more susceptible to adversarial threats, thus posing a significant security risk in resource-constrained environments. This finding has significant implications for the use of these models in real-world applications. As this work suggests, devices at the edge are more susceptible to adversarial attacks because they are more likely to run smaller models due to

Table 8 mAP of the baseline patches and the best patches generated in this work, at scale 0.20 (*faster R-CNN* [15], *advyolo* [31]), on the INRIA person dataset

	Patch (scale = 0.20)									
Model	*none*	*white*	*grey*	*random*	*faster R-CNN*	*advyolo*	*patch17*	*patch26*	*patch27*	*patch38*
v5n	100,00	62,10	64,08	76,44	61,21	58,42	48,51	52,01	49,42	36,76
v5s	100,00	68,11	69,66	77,65	56,49	51,19	47,54	50,81	47,70	45,99
v5m	100,00	76,23	74,34	79,06	70,22	60,80	61,21	58,95	56,04	59,52
v8n	100,00	67,94	68,04	77,43	63,34	52,59	52,08	53,73	52,46	48,31
v8s	100,00	74,81	74,91	79,08	58,74	53,75	60,92	59,00	54,95	58,13
v8m	100,00	76,77	76,52	77,67	68,26	51,15	65,74	63,37	61,26	65,50
v9t	100,00	72,53	74,09	79,38	71,27	66,17	65,90	63,79	61,81	60,37
v9s	100,00	75,13	75,91	80,96	68,29	62,45	62,18	59,36	53,55	57,85
v9m	100,00	77,42	76,70	78,54	67,41	51,87	67,01	65,25	61,52	65,63
v10n	100,00	72,93	75,58	81,53	49,15	59,08	46,19	48,24	46,17	39,20
v10s	100,00	76,80	77,45	80,01	59,37	51,73	43,36	41,16	38,23	39,94
v10m	100,00	76,62	77,62	80,25	70,01	53,94	57,82	55,10	51,98	48,65

Table 9 mAP of the baseline patches and the best patches generated in this work, at scale 0.22 (*faster R-CNN* [15], *advyolo* [31]), on the INRIA person dataset

	Patch (scale = 0.22)									
Model	*none*	*white*	*grey*	*random*	*faster R-CNN*	*advyolo*	*patch17*	*patch26*	*patch27*	*patch38*
v5n	100,00	49,82	50,12	69,82	46,50	47,14	26,39	30,60	30,95	19,90
v5s	100,00	60,52	60,55	73,35	39,86	41,37	30,73	32,47	28,83	31,84
v5m	100,00	73,20	66,57	75,87	59,08	55,42	50,76	50,20	46,74	48,25
v8n	100,00	58,45	57,92	73,86	51,27	43,60	36,43	37,25	33,96	31,27
v8s	100,00	70,37	67,66	74,95	45,99	43,11	45,95	43,29	37,78	41,50
v8m	100,00	72,11	69,49	73,95	58,58	42,37	58,97	56,77	54,94	59,95
v9t	100,00	69,58	70,29	75,81	64,56	54,60	51,03	48,12	46,64	41,30
v9s	100,00	72,28	71,69	78,60	61,47	53,41	44,42	42,38	35,51	42,49
v9m	100,00	73,36	70,54	76,09	56,07	44,01	59,54	56,96	51,30	58,17
v10n	100,00	68,23	71,24	77,88	37,81	48,56	30,01	33,47	29,18	24,51
v10s	100,00	73,55	74,16	78,47	47,61	39,30	30,87	27,03	24,66	27,22
v10m	100,00	72,97	72,24	77,41	61,47	45,18	52,65	48,46	45,58	40,16

Table 10 mAP of the baseline patches and the best patches generated in this work, at scale 0.20 (*faster R-CNN* [15], *advyolo* [31]), on the MPII pose human dataset

	Patch (scale = 0.20)									
Patch	*none*	*white*	*grey*	*random*	*faster R-CNN*	*advyolo*	*patch17*	*patch26*	*patch27*	*patch38*
v5n	100,00	35,74	34,39	61,92	35,54	31,54	23,39	25,29	24,22	16,77
v5s	100,00	47,02	44,33	62,20	30,75	30,65	24,82	25,37	22,55	23,51
v5m	100,00	65,35	58,38	69,23	51,66	46,01	46,82	46,03	43,05	44,67
v8n	100,00	44,26	40,65	69,21	33,70	27,08	23,94	23,92	24,41	19,52
v8s	100,00	58,78	55,12	68,08	37,12	34,50	35,24	33,05	28,71	31,42
v8m	100,00	67,18	63,04	69,68	52,59	45,21	53,37	52,91	50,92	54,51
v9t	100,00	60,29	59,33	73,63	47,91	37,73	33,72	31,39	29,83	25,75
v9s	100,00	63,02	57,89	72,98	46,36	39,35	34,85	31,39	27,04	32,20
v9m	100,00	66,21	59,08	69,00	47,48	42,46	50,57	50,31	46,44	50,60
v10n	100,00	53,64	55,04	76,78	27,20	33,28	20,06	21,80	19,64	14,83
v10s	100,00	65,78	64,02	75,26	36,76	33,11	30,40	26,51	23,40	26,54
v10m	100,00	69,45	66,03	75,43	57,54	49,38	50,56	48,98	45,33	44,41

Table 11 mAP improvement of the best patches generated in this work compared with the random patch and those generated with the *advyolo* method [31], at scale 0.20, on the INRIA person dataset. Darker green is better

	Compared with random patch				Compared with advyolo			
Model	*patch17*	*patch26*	*patch27*	*patch38*	*patch17*	*patch26*	*patch27*	*patch38*
v5n	36.5%	32.0%	35.3%	51.9%	17.0%	11.0%	15.4%	37.1%
v5s	38.8%	34.6%	38.6%	40.8%	7.1%	0.7%	6.8%	10.2%
v5m	22.6%	25.4%	29.1%	24.7%	-0.7%	3.0%	7.8%	2.1%
v8n	32.7%	30.6%	32.2%	37.6%	1.0%	-2.2%	0.2%	8.1%
v8s	23.0%	25.4%	30.5%	26.5%	-13.3%	-9.8%	-2.2%	-8.1%
v8m	15.4%	18.4%	21.1%	15.7%	-28.5%	-23.9%	-19.8%	-28.1%
v9t	17.0%	19.6%	22.1%	23.9%	0.4%	3.6%	6.6%	8.8%
v9s	23.2%	26.7%	33.9%	28.5%	0.4%	4.9%	14.3%	7.4%
v9m	14.7%	16.9%	21.7%	16.4%	-29.2%	-25.8%	-18.6%	-26.5%
v10n	43.3%	40.8%	43.4%	51.9%	21.8%	18.3%	21.9%	33.6%
v10s	45.8%	48.6%	52.2%	50.1%	16.2%	20.4%	26.1%	22.8%
v10m	28.0%	31.3%	35.2%	39.4%	-7.2%	-2.2%	3.6%	9.8%

Table 12 Performance comparison of model inference on the INRIA person dataset patched with *patch17*

	Edge AI device		**Workstation**	
Model	Inference time (ms)	Memory (MB)	Inference time (ms)	Memory (MB)
v5n	158.56	50.90	5.02	182.78
v5s	286.64	80.42	4.82	212.89
v5m	640.89	154.75	7.05	281.77
v8n	162.60	38.51	4.38	187.22
v8s	331.50	84.41	4.54	230.53
v8m	798.94	164.92	7.67	309.17
v9t	219.69	33.48	11.32	183.31
v9s	433.81	69.73	11.27	220.23
v9m	793.25	154.72	10.07	294.15
v10n	172.64	35.65	6.98	187.96
v10s	332.90	72.79	7.28	218.42
v10m	728.63	125.98	8.75	270.22

their computational limitations. These smaller models have lower generalisation capabilities, making them more likely to be fooled by adversarial patches. Although the mAP of the models remains consistent across different environments, the response time and the hardware resources used vary significantly. Table 12 shows the average response time and memory required to run the inference on the INRIA person dataset with the images patched with the *patch17* patch across different sizes of several YOLO models on the edge AI device selected in Table 2. The performance is compared with the response time and memory required to perform the same inference tasks on the workstation platform.

6 Conclusion

In this work we have successfully adapted the Ultralytics framework to enable its models to be used as detectors in the training phase of naturalistic adversarial patches in a state-of-the-art method. Adaptations such as the one implemented in this work can be developed for other patch generation methods, making the creation of adversarial patches for new object detectors more affordable and efficient. The source code developed in this work is freely available via public repositories for further research and evaluation.

We have also evaluated the effectiveness of naturalistic adversarial patches on the latest versions of Ultralytics YOLO object detectors. The experiments performed in this work were successful in creating adversarial patches for the YOLOv5, YOLOv8, YOLOv9 and YOLOv10 models using

YOLOv5, YOLOv8 and YOLOv9 as detectors in the training phase. To the best of our knowledge, these experiments represent one of the first documented attacks on a YOLOv8 model and the first documented attacks on YOLOv9 and YOLOv10 models based on naturalistic adversarial patches. These results highlight the potential vulnerabilities of current YOLO models, particularly small models designed to run on edge AI devices, which aim to achieve the same latency as cloud-based models running on more powerful servers.

While this work has focused on evaluating the vulnerability of YOLO detectors to naturalistic adversarial patches, future work should investigate effective defence mechanisms tailored to this class of attacks. One promising direction is adversarial training using patch-augmented datasets, where models are exposed to adversarial patches during training to increase their robustness. However, this approach is computationally intensive and may reduce generalisation if not carefully balanced. Detection-based defences, which aim to identify adversarial artefacts prior to inference, using statistical anomalies, saliency analysis, or frequency domain patterns, could be extended to specifically target localised and naturalistic patches. In parallel, model-level defences, such as robust feature extraction architectures or attention mechanisms that down-weight spurious regions, may provide improved resilience to spatially constrained perturbations. Furthermore, given the increasing importance of edge AI, future research should also explore lightweight, resource-efficient defences that operate within the hardware limitations of edge devices while maintaining detection latency and accuracy. In addition, a combination of multi-layered defences, from data to architecture to deployment, may be required to mitigate the evolving threat landscape posed by naturalistic adversarial patches.

Acknowledgements This work was supported by Project PID2021-124383OB-I00 of the Spanish National Plan for Research, Development and Innovation.

Funding Open Access funding provided thanks to the CRUE-CSIC agreement with Springer Nature.

Data availability The datasets used in this work are the INRIA person dataset [28] and the MPII human pose dataset [29], which are publicly available.

Declarations

Declaration of competing interest The authors declare that they have no known competing financial interests or personal relationships that could appear to have influenced the work reported in this paper.

Code availability The source code developed in this work is freely available via public GitHub repositories for further research and evaluation (see Section 4).

CRediT authorship contribution statement **D.L. Gala**: Data curation, Formal analysis, Investigation, Software, Validation, Visualization, Writing - original draft, Writing - review & editing. **J. Molleda**: Conceptualization, Methodology, Project administration, Resources, Supervision, Writing - original draft, Writing - review & editing. **R. Usamentiaga**: Funding acquisition, Resources, Writing - review & editing.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Biggio, B., Corona, I., Maiorca, D., Nelson, B., Šrndić, N., Laskov, P., Giacinto, G., Roli, F.: in Machine Learning and Knowledge Discovery. In: Blockeel, H., Kersting, K., Nijssen, S., Železný, F. (eds.) Databases, pp. 387–402. Springer, Berlin Heidelberg, Berlin, Heidelberg (2013)
2. Chen, S., Xue, M., Fan, L., Hao, S., Xu, L., Zhu, H., Li, B.: Computers & Security **73**, 326 (2018). https://doi.org/10.1016/j.cose.2017.11.007
3. Khosravy, M., Nakamura, K., Hirose, Y., Nitta, N., Babaguchi, N.: IEEE Transactions on Information Forensics and Security **17**, 357 (2022). https://doi.org/10.1109/TIFS.2022.3140687
4. Takemura, T., Yanai, N., Fujiwara, T.: Journal of Information Processing **28**, 1010–1024 (2020). https://doi.org/10.2197/ipsjjip.28.1010
5. Goodfellow, I.J., Shlens, J., Szegedy, C.: in *3rd International Conference on Learning Representations, ICLR 2015 - Conference Track Proceedings* (2015)
6. Madry, A., Makelov, A., Schmidt, L., Tsipras, D., Vladu, A.: (2018)
7. Carlini, N., Wagner, D.: in *Proceedings - IEEE Symposium on Security and Privacy* (2017), p. 39 - 57. https://doi.org/10.1109/SP.2017.49
8. Karmon, D., Zoran, D., Goldberg, Y.: in *35th International Conference on Machine Learning, ICML 2018*, vol. 6 (2018), vol. 6, p. 3903 - 3911
9. Su, J., Vargas, D.V., Sakurai, K.: IEEE Transactions on Evolutionary Computation **23**(5), 828 (2019). https://doi.org/10.1109/TEVC.2019.2890858
10. Su, W., Li, L., Liu, F., He, M., Liang, X.: Artificial Intelligence Review **55**(8), 6125–6183 (2022). https://doi.org/10.1007/s10462-022-10141-4
11. Entrialgo, J., Moreno-Vozmediano, R., Díaz, J.L.: Cluster Computing (2024). https://doi.org/10.1007/s10586-024-04359-w
12. Zhang, H., Ma, X.: Computers and Security **122** (2022). https://doi.org/10.1016/j.cose.2022.102876
13. Dhamija, L., Bansal, U.: Arabian Journal for Science and Engineering **49**, 13203 (2024). https://doi.org/10.1007/s13369-024-09293-y
14. Qin, Y., Zhang, K., Pan, H.: Computers and Security **134** (2023). https://doi.org/10.1016/j.cose.2023.103460

15. Hu, Y.C.T., Kung, B.H., Tan, D.S., Chen, J.C., Hua, K.L., Cheng, W.H.: IEEE/CVF International Conference on Computer Vision (ICCV) (2021). https://doi.org/10.1109/ICCV48922.2021.00775
16. Biggio, B., Roli, F.: Pattern Recognition **84**, 317 (2018). https://doi.org/10.1016/j.patcog.2018.07.023
17. Wang, X., Li, J., Kuang, X., an Tan, Y., Li, J.: Journal of Parallel and Distributed Computing **130**, 12 (2019). https://doi.org/10.1016/j.jpdc.2019.03.003
18. Martins, N., Cruz, J.M., Cruz, T., Henriques Abreu, P.: IEEE Access **8**, 35403 (2020). https://doi.org/10.1109/ACCESS.2020.2974752
19. Qayyum, A., Usama, M., Qadir, J., Al-Fuqaha, A.: IEEE Communications Surveys & Tutorials **22**(2), 998 (2020). https://doi.org/10.1109/COMST.2020.2975048
20. Machado, G.R., Silva, E., Goldschmidt, R.R.: ACM Computing Surveys (CSUR) **55**(1), 1 (2021). https://doi.org/10.1145/3485133
21. Bountakas, P., Zarras, A., Lekidis, A., Xenakis, C.: Computer Science Review **49**, 100573 (2023). https://doi.org/10.1016/j.cosrev.2023.100573
22. Yang, D.Y., Xiong, J., Li, X., Yan, X., Raiti, J., Wang, Y., Wu, H., Zhong, Z., in,: 9th IEEE Annual Ubiquitous Computing. Electronics & Mobile Communication Conference (UEMCON) **2018**, 368–374 (2018). https://doi.org/10.1109/UEMCON.2018.8796670
23. Choi, J.I., Tian, Q.: in *2022 IEEE Intelligent Vehicles Symposium (IV)* (2022), pp. 1011–1017. https://doi.org/10.1109/IV51971.2022.9827222
24. Qin, Y., Zhang, K., Pan, H.: Computers & Security **134**, 103460 (2023). https://doi.org/10.1016/j.cose.2023.103460
25. Lin, C.Y., Huang, T.Y., Ng, H.F., Lin, W.Y., Farady, I.: IEEE Access **12**, 32772 (2024). https://doi.org/10.1109/ACCESS.2024.3371507
26. Ding, Z., Sun, L., Mao, X., Dai, L., Xu, B.: Signal. Image and Video Processing **18**(3), 2485–2497 (2024). https://doi.org/10.1007/s11760-023-02924-1. Cited by: 0
27. Deng, J., Dong, W., Socher, R., Li, L.J., Li, K., Fei-Fei, L.: in *2009 IEEE conference on computer vision and pattern recognition* (Ieee, 2009), pp. 248–255. https://doi.org/10.1109/CVPR.2009.5206848
28. Dalal, N., Triggs, B.: in *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)*, vol. 1 (2005), vol. 1, pp. 886–893 vol. 1. https://doi.org/10.1109/CVPR.2005.177
29. Andriluka, M., Pishchulin, L., Gehler, P., Schiele, B.: in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2014)
30. Everingham, M., Van Gool, L., Williams, C.K.I., Winn, J., Zisserman, A.: International Journal of Computer Vision **88**(2), 303–338 (2010). https://doi.org/10.1007/s11263-009-0275-4
31. Thys, S., Ranst, W., Goedeme, T.: in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)* (IEEE Computer Society, Los Alamitos, CA, USA, 2019), pp. 49–55. https://doi.org/10.1109/CVPRW.2019.00012
32. Wang, A., Chen, H., Liu, L., Chen, K., Lin, Z., Han, J.: G. Ding. Yolov10: Real-time end-to-end object detection (2024). https://arxiv.org/abs/2405.14458

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.